

1. WHAT IS MATPLOTLIB ?

Matplotlib is a powerful Python library used for data visualization. It is used to create static, animated and interactive plots in Python. It is the most popular library for plotting graphs in Python.

2. INSTALLATION

We can install matplotlib using pip.

```
pip install matplotlib
```

Note:

Make sure you have Python installed in your system.

3. IMPORTING MATPLOTLIB

The most commonly used module in matplotlib is pyplot.

```
import matplotlib.pyplot as plt
```

plt is an alias which is commonly used.

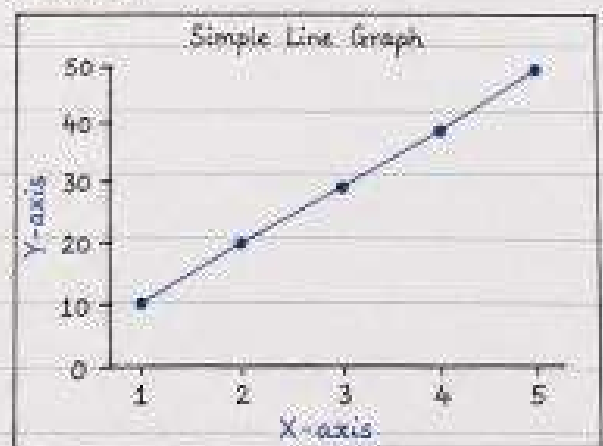
4. FIRST GRAPH

Let's create our first simple line graph.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]
plt.plot(x, y)
plt.show()
```

OUTPUT :



EXPLANATION:

- x and y are the data points.
- `plt.plot(x, y)` is used to draw the line.
- `plt.show()` is used to display the graph.

KEY TAKEAWAYS :

- Matplotlib is used for data visualization.
- pyplot module is used to create graphs.
- `plt.plot()` is used to plot line graphs.
- Always use `plt.show()` to display the graph.

NEXT TOPIC →

Line Chart with Title,
X label and Y label.
(Matplotlib Notes - 2)

MATPLOTLIB NOTES - 2

Page No. : 2

Date : 26 / 05 / 2024

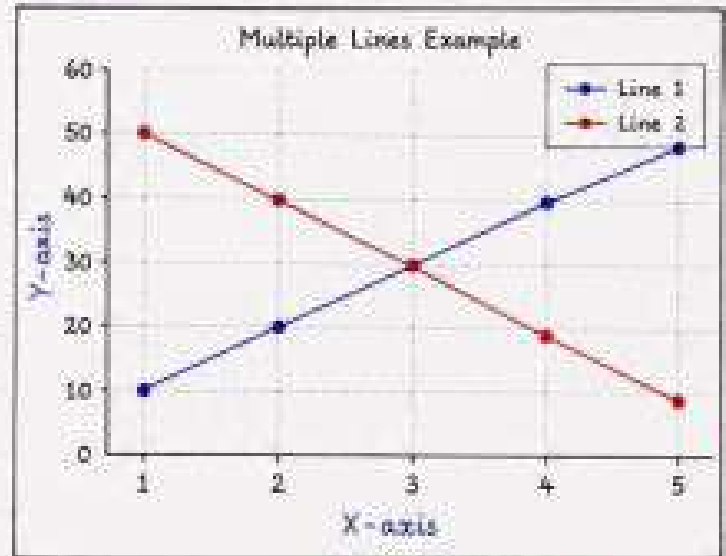
5. MULTIPLE LINES IN ONE GRAPH

We can plot more than one line on the same graph to compare multiple data sets.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 30, 40, 50]
y2 = [50, 40, 30, 20, 10]
plt.plot(x, y1, label = 'Line 1')
plt.plot(x, y2, label = 'Line 2')
plt.title('Multiple Lines Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend() # show legend
plt.grid(True)
plt.show()
```

OUTPUT :



EXPLANATION :

- We plotted two lines (y1 and y2) on the same graph.
- `label` is used to name the lines.
- `plt.legend()` is used to show the legend (names of lines).
- `plt.grid(True)` adds grid lines to the graph.

TIP :

Legends are useful when we have multiple lines or plots to understand which line represents which data. 😊

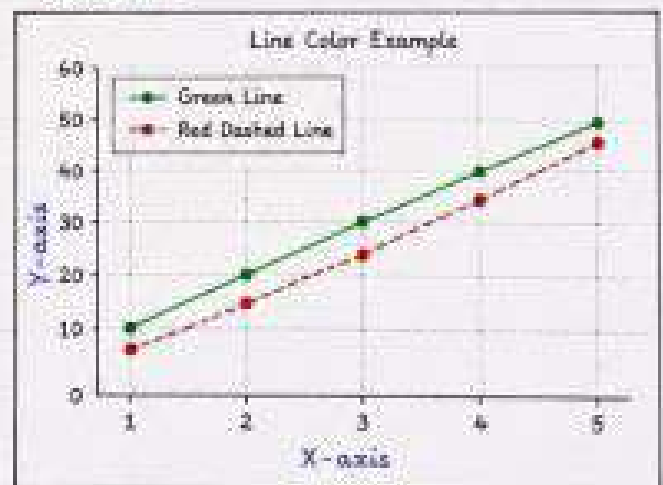
6. LINE COLOR

We can change the color of lines in our plot.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]
plt.plot(x, y, color = 'green', label = 'Green Line')
plt.plot(x, y, color = 'red', linestyle = '--',
         label = 'Red Dashed Line')
plt.title('Line Color Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT :



EXPLANATION :

- `color` parameter is used to set the color of the line.
- `linestyle = '--'` makes the line dashed.
- We can use many color names or codes.

COMMON COLOR NAMES :

'red', 'green', 'blue', 'black', 'orange', 'purple', 'brown', 'pink', 'gray' etc.

7. MARKERS IN LINE PLOT

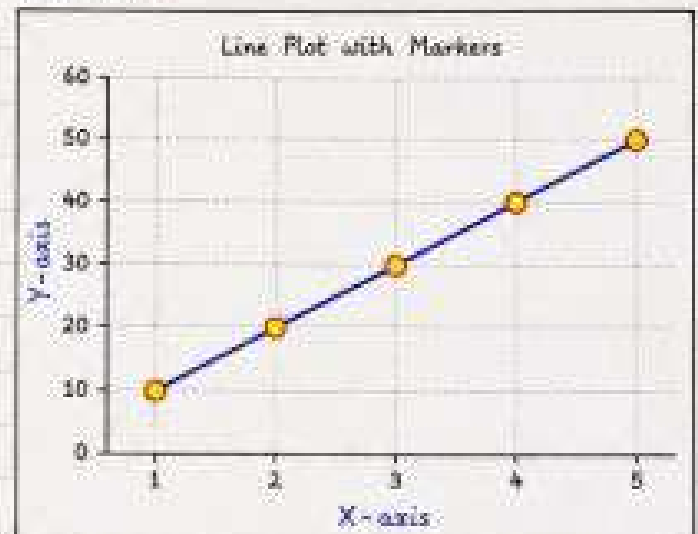
We can use markers to highlight each data point in the line.

Markers make the graph more visible and attractive.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]
plt.plot(x, y, marker='o', color='b',
         markersize=8, markeredgecolor='r',
         markerfacecolor='yellow', linewidth=2)
plt.title('Line Plot with Markers')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.show()
```

OUTPUT :



EXPLANATION :

- **marker = 'o'** uses circle marker on each point.
- **color = 'b'** sets the line color to blue.
- **markersize** controls the size of the marker.
- **markeredgecolor** sets the edge color of the marker.
- **markerfacecolor** sets the fill color of the marker.
- **linewidth** sets the thickness of the line.

COMMON MARKERS

'o' → Circle	'v' → Triangle Down
'*' → Star	'D' → Diamond
's' → Square	'+' → Plus
'^' → Triangle Up	'x' → Cross

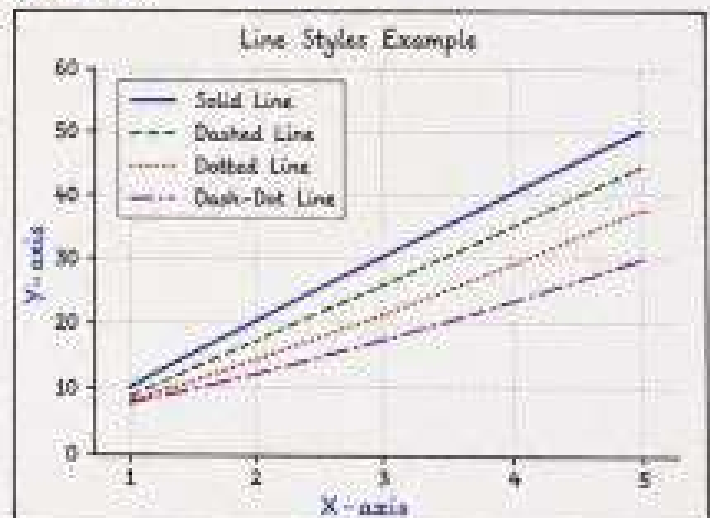
8. LINE STYLE

We can change the style of the line using linestyle parameter.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 30, 40, 50]
plt.plot(x, y, linestyle='-', label='Solid Line')
plt.plot(x, y, linestyle='--', label='Dashed Line')
plt.plot(x, y, linestyle=':', label='Dotted Line')
plt.plot(x, y, linestyle='-.', label='Dash-Dot Line')
plt.title('Line Styles Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT :



EXPLANATION :

- **linestyle** controls the style of the line.
- **'-'** Solid line (default)
- **'--'** Dashed line
- **'.'** Dotted line
- **'-.'** Dash-Dot line

NOTE :

We can combine color, marker and line style together to make our plots more effective and easy to understand. 😊

9. BAR CHART (VERTICAL) - `plt.bar()`

We can create bar charts using `plt.bar()`. It is used to compare discrete categories.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
categories = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales = [20, 35, 30, 45, 40]
plt.bar(categories, sales, color = 'skyblue',
        edgcolor = 'black', width = 0.6)
plt.title('Monthly Sales Bar Chart')
plt.xlabel('Month')
plt.ylabel('Sales (in 1000s)')
plt.grid(axis = 'y', linestyle = '--', alpha = 0.7)
plt.show()
```

OUTPUT :



EXPLANATION :

- `plt.bar()` is used to create vertical bar chart.
- `categories` are shown on the x-axis and sales on the y-axis.
- `color` sets the fill color of bars.
- `edgcolor` gives border color to bars.
- `width` controls the width of bars.
- `grid(axis = 'y')` adds horizontal grid lines.

NOTE :

Bar chart is best for comparing values across different categories.

10. HORIZONTAL BAR CHART - `plt.barh()`

We can create horizontal bar chart using `plt.barh()`.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
products = ['Laptop', 'Mobile', 'Tablet', 'Smartwatch']
sales = [550, 750, 300, 450]
plt.barh(products, sales, color = 'lightgreen',
        edgcolor = 'black')
plt.title('Product Sales (Horizontal Bar Chart)')
plt.xlabel('Sales (in units)')
plt.ylabel('Products')
plt.grid(axis = 'x', linestyle = '--', alpha = 0.7)
plt.show()
```

OUTPUT :



EXPLANATION :

- `plt.barh()` is used to create horizontal bar chart.
- `categories` are shown on the y-axis and values on the x-axis.
- It is useful when category names are long.

KEY TAKEAWAYS :

- ✓ `plt.bar()` → Vertical bar chart
- ✓ `plt.barh()` → Horizontal bar chart
- ✓ Both are useful for comparing categorical data.

COMMON PARAMETERS :

- `color` : Sets the color of bars.
- `edgcolor` : Sets border color of bars.
- `width` : Sets width of bars.
- `xlabel()`, `ylabel()` : Sets axis labels.
- `title()` : Sets the title of the chart.
- `grid()` : Adds grid lines.

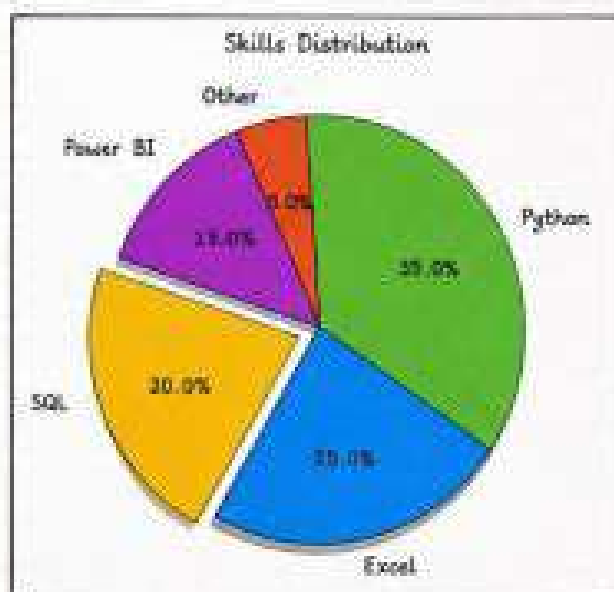
11. PIE CHART - `plt.pie()`

We can create pie charts using `plt.pie()`. It is used to show part-to-whole relationship.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
labels = ['Python', 'Excel', 'SQL', 'Power BI', 'Other']
sizes = [35, 25, 20, 15, 5]
colors = ['#4CAF50', '#2196F3', '#FFC107',
          '#9C27B0', '#FF5722']
plt.pie(sizes, labels = labels, colors = colors,
        autopct = '%1.1f%%', startangle = 90,
        shadow = True, explode = (0, 0, 0.1, 0, 0))
plt.title('Skills Distribution')
plt.axis('equal') # Equal aspect ratio ensures
                  # circle shape
plt.show()
```

OUTPUT :



EXPLANATION :

- labels** : Names shown for each slice.
- sizes** : Values for each category.
- colors** : Colors for the slices.
- autopct** : Shows percentage on each slice.
- startangle** : Sets the starting angle of the pie chart.
- shadow** : Adds a shadow effect to the pie chart.
- explode** : Separates (explodes) the slice(s).
- axis('equal')** : Ensures pie chart is drawn as a circle (not oval).

TIP :

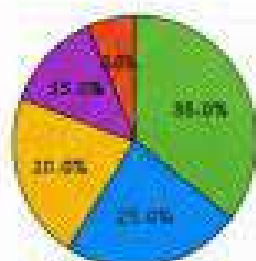
Pie charts are best used when you want to show the percentage contribution of categories in whole.



OTHER USEFUL OPTIONS IN PIE CHART

1. WITH PERCENTAGE

```
plt.pie(sizes, labels = labels,
        autopct = '%1.1f%%')
```



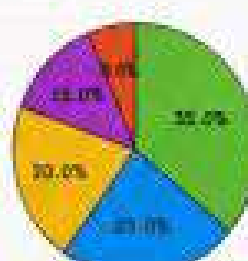
2. EXPLODE MULTIPLE SLICES

```
explode = (0.1, 0, 0.1,
           0, 0.1)
```



3. WITHOUT LABELS

```
plt.pie(sizes, labels = None,
        autopct = '%1.1f%%')
```



4. DONUT CHART

```
plt.pie(sizes, labels = labels,
        wedgeprops = {'width': 0.4},
        autopct = '%1.1f%%')
```



KEY TAKEAWAYS :

- ✓ `plt.pie()` is used to create pie charts.
- ✓ `autopct` shows percentage of each slice.
- ✓ `startangle` rotates the pie chart.
- ✓ `explode` is used to highlight important slice(s).
- ✓ `axis('equal')` keeps the pie chart in circular shape.

WHEN TO USE PIE CHART ?

- When you want to show parts of a whole.
- When total number of categories is small (preferably less than 6 or 7).
- Example : Market share, Budget allocation, Survey results, etc. 😊

12. HISTOGRAM - `plt.hist()`

We can create histogram to show the distribution of a continuous data. It groups the data into bins (ranges).

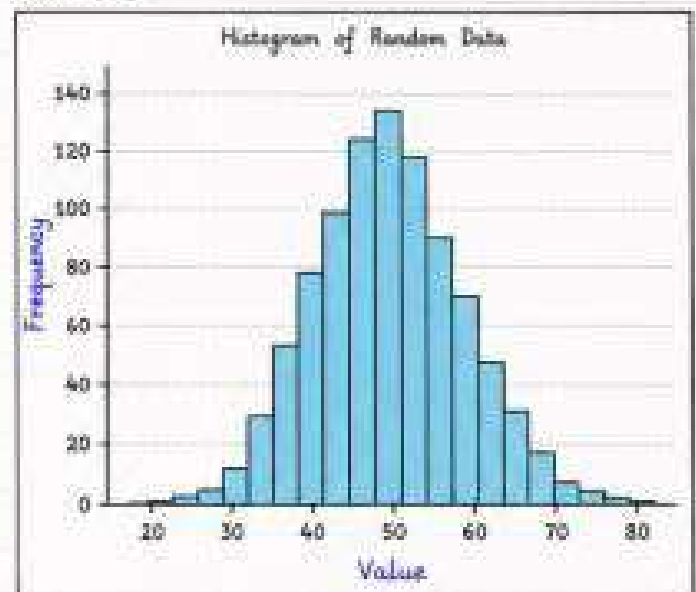
EXAMPLE CODE :

```
import matplotlib.pyplot as plt
import numpy as np

# Generate some random data
np.random.seed(0) # for reproducibility
data = np.random.normal(loc = 50, scale = 10, size = 1000)

# Plot histogram
plt.hist(data, bins = 15, color = 'skyblue',
         edgecolor = 'black')
plt.title('Histogram of Random Data')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(axis = 'y', linestyle = '--', alpha = 0.7)
plt.show()
```

OUTPUT :



EXPLANATION :

- `data` : Random data generated using normal distribution.
- `bins = 15` : Data is divided into 15 equal-width bins.
- `color` : Fill color of the bars.
- `edgecolor` : Border color of the bars.
- `grid(axis = 'y')` : Adds horizontal grid lines.

Tip :

Histogram is very useful to understand the distribution and pattern of data.



13. SCATTER PLOT - `plt.scatter()`

We can create scatter plots to show relationship between two continuous variables.

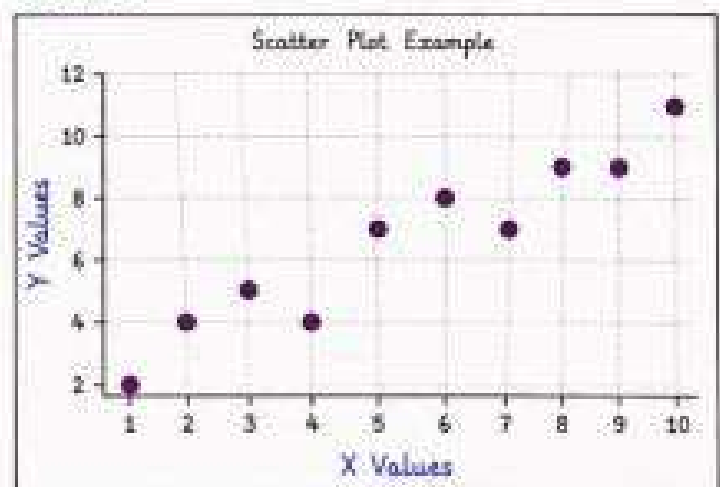
EXAMPLE CODE :

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y = np.array([2, 4, 5, 4, 7, 8, 7, 9, 9, 11])

# Scatter plot
plt.scatter(x, y, color = 'purple', marker = 'o', s = 80)
plt.title('Scatter Plot Example')
plt.xlabel('X Values')
plt.ylabel('Y Values')
plt.grid(True, linestyle = '--', alpha = 0.6)
plt.show()
```

OUTPUT :



EXPLANATION :

- `plt.scatter()` is used to create scatter plot.
- `marker = 'o'` sets circular markers.
- `s = 80` controls the size of markers.
- Grid helps to read the values easily.

COMMON MARKERS IN SCATTER PLOT :

'o'	→ Circle	'v'	→ Triangle Down
'*''	→ Star	'D'	→ Diamond
's'	→ Square	'+''	→ Plus
'^'	→ Triangle Up	'x'	→ Cross

14. GRID IN PLOTS - `plt.grid()`

We can add grid lines to the plot to make it easier to read values.

EXAMPLE CODE :

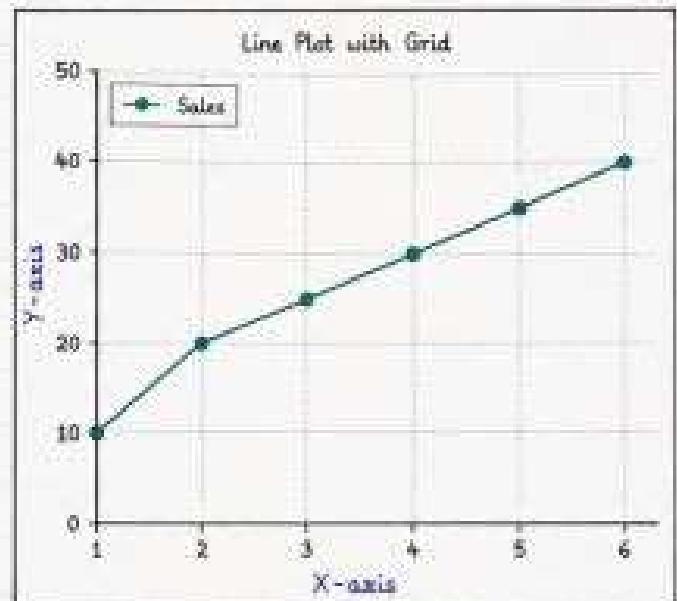
```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5, 6]
y = [10, 20, 25, 30, 35, 40]

# Plot line
plt.plot(x, y, marker='o', color='teal',
         linewidth=2, label='Sales')

# Add grid
plt.grid(True, linestyle='--', color='gray',
        linewidth=0.7)

plt.title('Line Plot with Grid')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

OUTPUT :



EXPLANATION :

- `plt.grid(True)` → Turns on the grid.
- `linestyle='--'` → Makes the grid dashed.
- `color='gray'` → Sets the grid line color.
- `linewidth=0.7` → Sets the thickness of grid lines.

TIP :

Grid lines are very helpful when we want to match values on both axes.



15. LEGEND IN PLOTS - `plt.legend()`

We can add a legend to the plot to identify different lines, bars, etc.

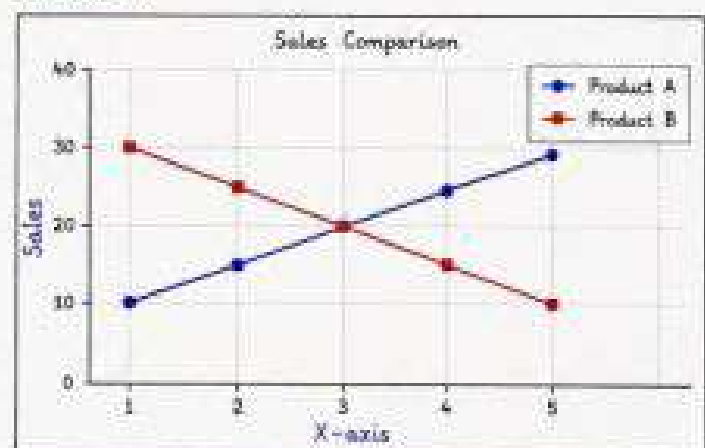
EXAMPLE CODE :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [10, 15, 20, 25, 30]
y2 = [30, 25, 20, 15, 10]

plt.plot(x, y1, color='blue', marker='x',
        linewidth=2, label='Product A')
plt.plot(x, y2, color='red', marker='x',
        linewidth=2, label='Product B')

plt.title('Sales Comparison')
plt.xlabel('X-axis')
plt.ylabel('Sales')
plt.legend() # show legend
plt.show()
```

OUTPUT :



EXPLANATION :

- `label` → Used inside `plot()` to name each line.
- `plt.legend()` → Displays the legend on the plot.

LEGEND POSITION OPTIONS :

'best'	→ Default (best position)	'lower right'	→ Bottom right
'upper right'	→ Top right	'center right'	→ Middle right
'upper left'	→ Top left	'center left'	→ Middle left
'lower left'	→ Bottom left	'center'	→ Center of plot

KEY TAKEAWAYS :

- ✓ `plt.grid()` → Adds grid lines to the plot for better readability.
- ✓ `plt.legend()` → Shows a box that describes lines/plots in the graph.
- ✓ These features make our plots more professional and easy to understand.



16. SUBPLOTS - `plt.subplot()`

We can create multiple plots in a single figure using subplots.
It is very useful to compare different data side by side.

EXAMPLE CODE :

```
import matplotlib.pyplot as plt

# Data for plots
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 30, 40, 50]
y2 = [50, 40, 30, 20, 10]
y3 = [5, 10, 15, 20, 25]

# Create subplots (2 rows, 2 columns)
plt.subplot(2, 2, 1)      # Row 1, Col 1
plt.plot(x, y1, color='blue', marker='o')
plt.title('Plot 1')
plt.xlabel('X')
plt.ylabel('Y')

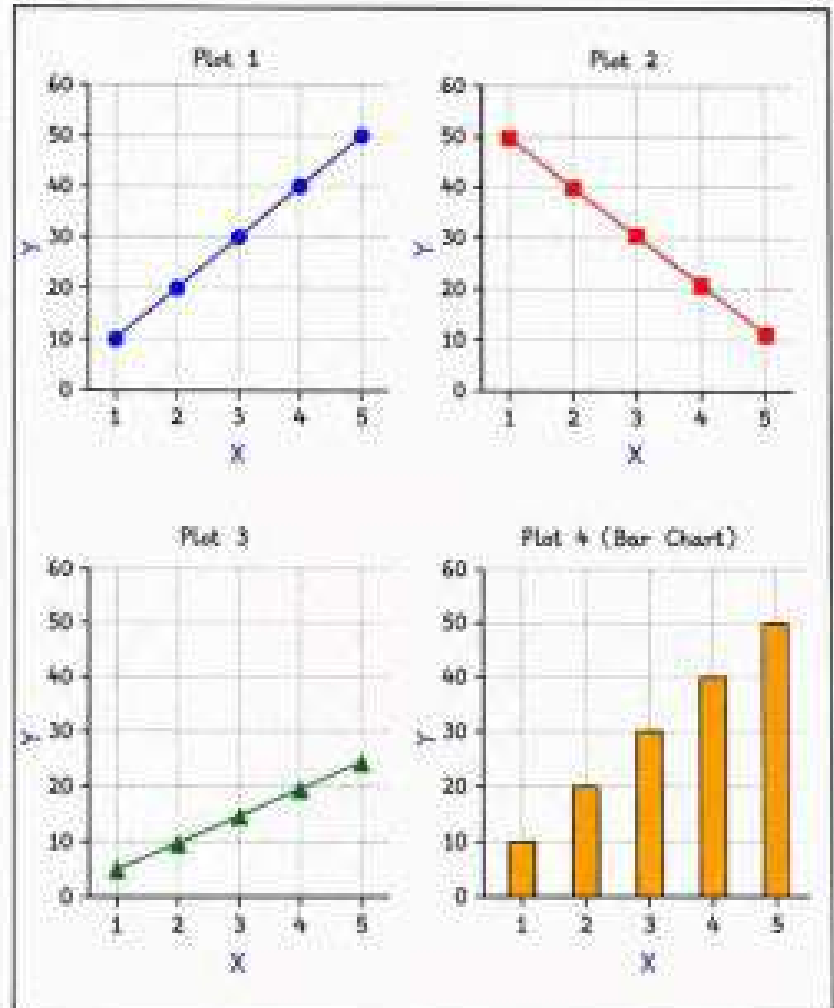
plt.subplot(2, 2, 2)      # Row 1, Col 2
plt.plot(x, y2, color='red', marker='x')
plt.title('Plot 2')
plt.xlabel('X')
plt.ylabel('Y')

plt.subplot(2, 2, 3)      # Row 2, Col 1
plt.plot(x, y3, color='green', marker='^')
plt.title('Plot 3')
plt.xlabel('X')
plt.ylabel('Y')

plt.subplot(2, 2, 4)      # Row 2, Col 2
plt.bar(x, y3, color='orange')
plt.title('Plot 4 (Bar Chart)')
plt.xlabel('X')
plt.ylabel('Y')

plt.tight_layout()        # Adjust spacing
plt.show()
```

OUTPUT :



SUBPLOT SYNTAX :

`plt.subplot(nrows, ncols, index)`

- `nrows` → Number of rows
- `ncols` → Number of columns
- `index` → Position of the plot (starts from 1)



EXPLANATION :

- `plt.subplot(2, 2, 1)` means 2 rows, 2 columns and this is the 1st plot
- We can add different types of plots in each subplot.
- `plt.tight_layout()` adjusts spacing between plots automatically.

OTHER USEFUL FUNCTIONS :

- `plt.subplots()` → Object-oriented way to create subplots.
- `figsize=(w, h)` → Set the size of the figure.
- `sharex=True` → Share x-axis among plots.
- `sharey=True` → Share y-axis among plots.

KEY TAKEAWAYS :

- ✓ Subplots help to display multiple plots together.
- ✓ It is easy to compare different data.
- ✓ We can combine line, bar, scatter, etc. in subplots.
- ✓ Always use `tight_layout()` for a neat look.



17. FIGURE SIZE - `plt.figure(figsize=())`

We can change the size of the entire figure (width, height) using `figsize` parameter in `plt.figure()`.

EXAMPLE CODE :

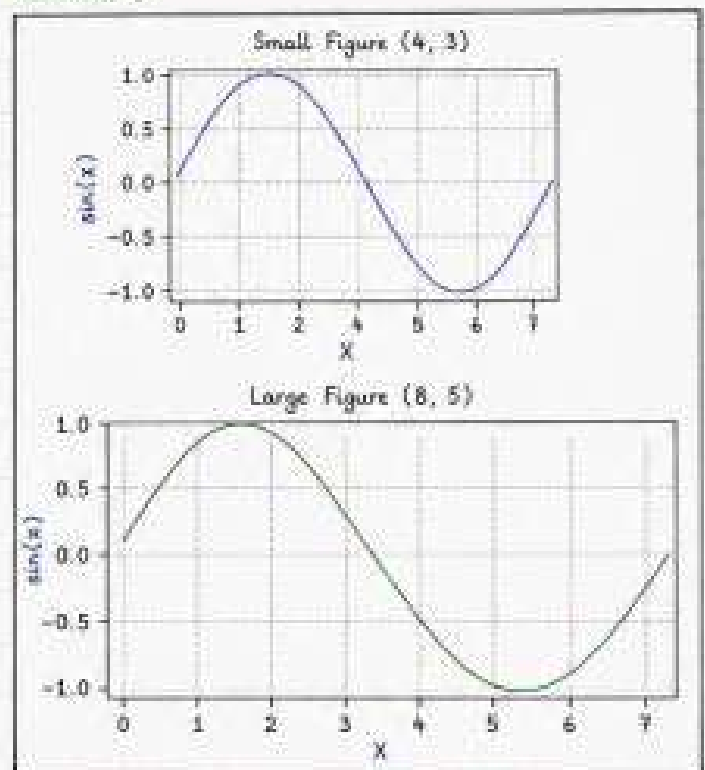
```
import matplotlib.pyplot as plt
import numpy as np

# Data
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

# Small figure
plt.figure(figsize=(4, 3))
plt.plot(x, y, color='blue')
plt.title('Small Figure (4, 3)')
plt.xlabel('X')
plt.ylabel('sin(x)')
plt.grid(True, linestyle='--', alpha=0.4)

# Large figure
plt.figure(figsize=(8, 5))
plt.plot(x, y, color='green')
plt.title('Large Figure (8, 5)')
plt.xlabel('X')
plt.ylabel('sin(x)')
plt.grid(True, linestyle='--', alpha=0.4)
plt.show()
```

OUTPUT :



EXPLANATION :

- `plt.figure(figsize=(width, height))` sets the size of the figure in inches.
- The first plot has size (4, 3) → smaller.
- The second plot has size (8, 5) → bigger.

NOTE :

Use `figsize` to make your plots more suitable for presentations or reports.

18. SAVE FIGURE - `plt.savefig()`

We can save our plot as an image file using `plt.savefig()`.

EXAMPLE CODE :

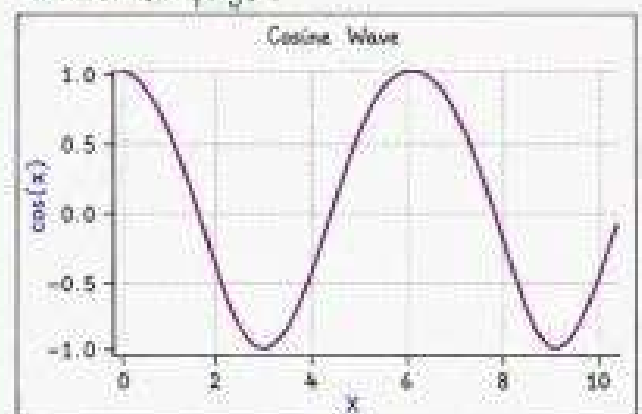
```
import matplotlib.pyplot as plt
import numpy as np

# Data
x = np.linspace(0, 10, 100)
y = np.cos(x)

# Plot
plt.plot(x, y, color='purple', linewidth=2)
plt.title('Cosine Wave')
plt.xlabel('X')
plt.ylabel('cos(x)')
plt.grid(True, linestyle='--', alpha=0.4)

# Save the figure
plt.savefig('cosine_wave.png', dpi=300, bbox_inches='tight')
# dpi → image quality
plt.show()
```

OUTPUT (Display) :



File saved as : cosine_wave.png

EXPLANATION :

- `plt.savefig('filename.png')` saves the current figure.
- `dpi=300` → sets high resolution (dots per inch).
- `bbox_inches='tight'` → removes extra white spaces.
- The file is saved in the current working directory.

COMMON FILE FORMATS :

- PNG → Good for presentations (high quality)
- JPG → Good for web (smaller size)
- PDF → Best for documents (vector format)
- SVG → Scalable vector graphics (vector format)

High dpi
= Better
Quality

19. LINE STYLES, COLORS & MARKERS

We can customize the appearance of lines using different line styles, colors and markers.

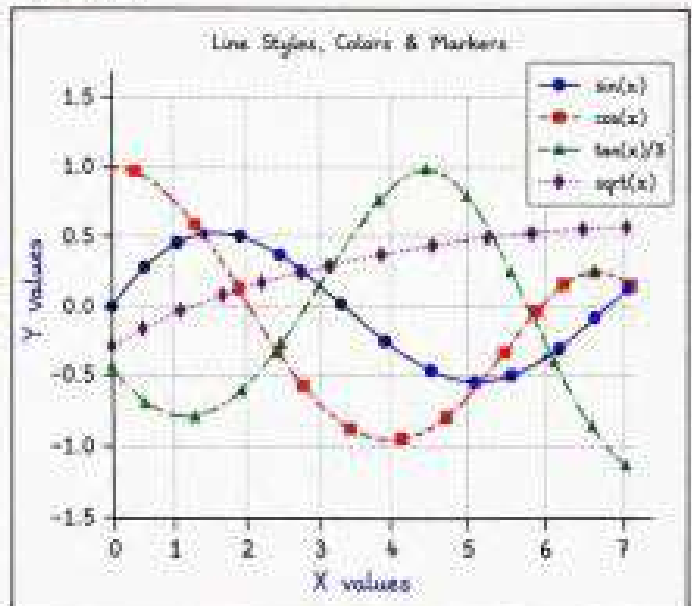
EXAMPLE CODE :

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x) / 3 # scaled for better view
y4 = np.sqrt(x)

# Plot with different line styles, colors and markers
plt.plot(x, y1, color='blue', linestyle='-', marker='o', label='sin(x)')
plt.plot(x, y2, color='red', linestyle='--', marker='s', label='cos(x)')
plt.plot(x, y3, color='green', linestyle='-.', marker='^', label='tan(x)/3')
plt.plot(x, y4, color='purple', linestyle=':', marker='d', label='sqrt(x)')

plt.title('Line Styles, Colors & Markers')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.legend() # show legend
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

OUTPUT :



TIP :

You can mix and match styles, colors and markers to make your plots more informative and attractive.

COMMON LINE STYLES :

- '-' Solid line
- '--' Dashed line
- '-.' Dash-dot line
- ':' Dotted line

COMMON MARKERS :

- 'o' Circle
- 's' Square
- '^' Triangle Up
- 'v' Triangle Down
- 'D' Diamond
- '*' Star
- 'x' X (Cross)
- '+' Plus

20. MULTIPLE PLOTS (SUBPLOTS) - plt.subplots()

We can create multiple plots in one figure using plt.subplots().

EXAMPLE CODE :

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x) / 3
y4 = np.sqrt(x)

# Create figure and 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 7))

axes[0, 0].plot(x, y1, color='blue')
axes[0, 0].set_title('sin(x)')
axes[0, 0].set_ylabel('Y')
axes[0, 0].grid(True, linestyle='--', alpha=0.6)

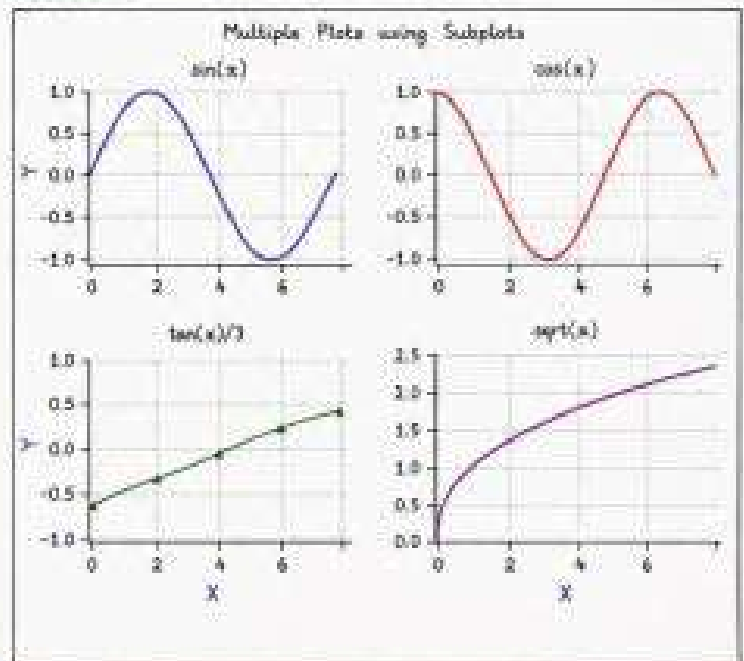
axes[0, 1].plot(x, y2, color='red')
axes[0, 1].set_title('cos(x)')
axes[0, 1].grid(True, linestyle='--', alpha=0.6)

axes[1, 0].plot(x, y3, color='green')
axes[1, 0].set_title('tan(x)/3')
axes[1, 0].set_ylabel('Y')
axes[1, 0].grid(True, linestyle='--', alpha=0.6)

axes[1, 1].plot(x, y4, color='purple')
axes[1, 1].set_title('sqrt(x)')
axes[1, 1].grid(True, linestyle='--', alpha=0.6)

fig.suptitle('Multiple Plots using Subplots', fontsize=14)
fig.tight_layout(rect=[0, 0, 1, 0.95]) # adjust layout
plt.show()
```

OUTPUT :



KEY TAKEAWAYS :

- ✓ plt.subplots(nrows, ncols) creates a grid of subplots.
- ✓ axes is an array of Axes-objects.
- ✓ We can customize each subplot individually.
- ✓ fig.tight_layout() automatically adjusts spacing.



IMPORTANT NOTES :

- Indices start from 0. Example: axes[0, 1] → first row, second column.
- You can use figsize=(width, height) to set figure size.
- Use suptitle() for main title of the entire figure.
- Subplots are very useful for comparing multiple data sets.

